mini-SWE-agent 동작 구조 및 워크플로우 (Workflow)

이 문서는 mini-SWE-agent의 시스템 동작 원리, 핵심 컴포넌트 아키텍처, 그리고 실행 제어 흐름(Workflow)을 상세히 설명합니다.

---

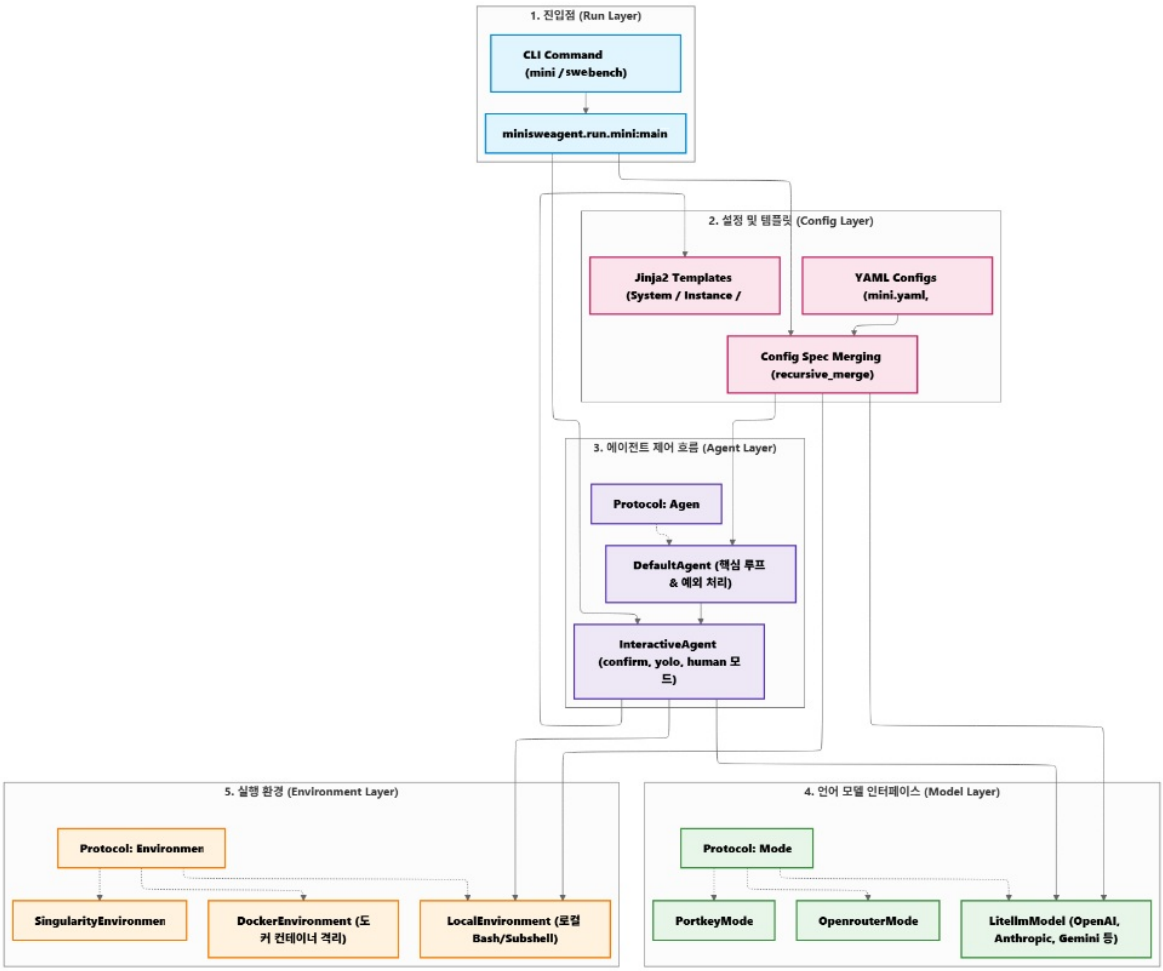
1. 프로젝트 개요

mini-SWE-agent는 GitHub 이슈 및 소프트웨어 엔지니어링 문제를 자율적으로 해결하는 초경량 AI 코딩 에이전트입니다. 단순성, 최소주의, 다형성(Polymorphism)을 원칙으로 설계되어 있으며 다음 5개의 핵심 레이어로 구성됩니다:

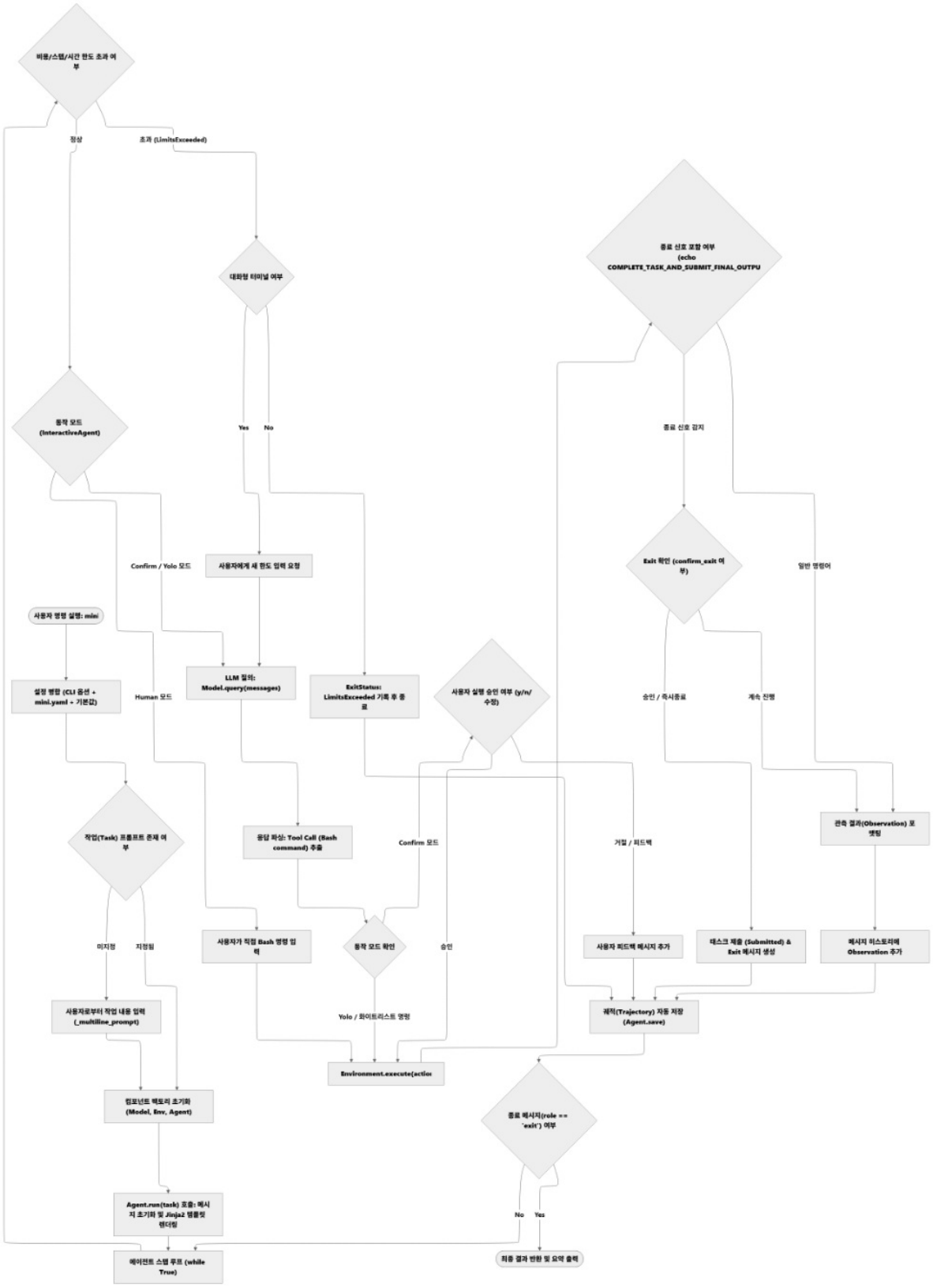
1. **Run (진입점)**: CLI 명령(`mini` , `swebench` 등)을 파싱하고 환경과 에이전트를 초기화하여 시작합니다.
2. **Agent (에이전트 제어 흐름)**: 프롬프트 렌더링, LLM 질의, 도구 실행, 인터랙션 모드, 예외 처리 루프를 관리합니다.
3. **Model (LLM 인터페이스)**: LiteLLM, OpenRouter, Portkey 등을 통해 LLM에 질의하고 Tool Call 응답 및 관측(Observation) 메시지를 포맷팅합니다.
4. **Environment (실행 환경)**: 에이전트가 생성한 셸 명령어를 로컬(`Local`), 컨테이너(`Docker`), 샌드박스(`Singularity`) 등 격리된 환경에서 실행합니다.
5. **Config (설정 및 템플릿)**: 계층적 YAML 설정 병합, Jinja2 기반 템플릿 렌더링, 상태 직렬화(Trajectory)를 지원합니다.

---

2. 컴포넌트 아키텍처 (Mermaid)



3. 전체 실행 흐름도 (Execution Flowchart)



3.1 'mini' 명령어 실행 시 상세 워크플로우 (Step-by-Step Lifecycle)

사용자가 터미널에서 `mini` (또는 `mini -m <model> -t <task>`) 명령어를 실행했을 때 내부적으로 처리되는 전체 라이프사이클과 단계별 흐름입니다:

1단계: 패키지 로딩 및 전역 설정 초기화 (`minisweagent/__init__.py`)

- `mini = "minisweagent.run.mini:app"` mini.py
- **버전 및 로딩 배너 출력**: 콘솔에 `mini-swe-agent` 버전 정보 및 로딩할 전역 설정 파일 경로를 안내합니다.
- **전역 설정 디렉토리 확보**:
  - `MSWEA_GLOBAL_CONFIG_DIR` 환경변수 또는 OS 표준 설정 경로(`platformdirs.user_config_dir`)를 참조합니다.
  - Windows 기준: `C:\Users\<사용자>\AppData\Local\mini-swe-agent` 디렉토리를 자동 생성합니다.
- **전역 환경변수 로드**: 전역 설정 디렉토리 내의 `.env` 파일(API Key 등 보관)을 `dotenv.load_dotenv()` 로 불러옵니다.

2단계: 첫 실행 마법사 및 설정 병합 (`minisweagent.run.mini:main`)

- **최초 실행 확인 (`configure_if_first_time`)**:
  - 전역 설정 파일(`.env`)에 모델 또는 API 키 설정이 없으면 사용자에게 대화형 마법사(모델 선택 및 API 키 입력 폼)를 제공합니다.
- **계층적 설정 병합 (`get_config_from_spec` & `recursive_merge`)**:
  - 기본 설정 파일(mini.yaml)을 파싱합니다.
  - 사용자가 추가 전달한 `-c / --config` 파일이 있다면 이를 파싱합니다.
  - CLI 명령행 인자(`-m/--model` , `-y/--yolo` , `-l/--cost-limit` , `-o/--output` 등)를 추출합니다.
  - recursive_merge 함수로 우선순위에 맞게 병합하여 최종 통합 딕셔너리(`config`)를 생성합니다.

3단계: 작업(Task) 확인 및 사용자 프롬프트 입력

- **태스크 파라미터 확인**:
  - CLI 옵션 `-t / --task "<과제 내용>"` 이 제공되었는지 검사합니다.
- **대화형 입력 프롬프트 (`_multiline_prompt`)**:
  - 태스크가 지정되지 않은 경우 터미널에 `What do you want to do?` 를 출력하고 사용자가 해결하고자 하는 문제나 지시사항을 여러 줄로 입력할 수 있도록 프롬프트를 엽니다.

4단계: 3대 핵심 컴포넌트 팩토리 초기화

통합 `config` 를 바탕으로 다형성(Polymorphism) 구조를 가진 3대 객체를 생성합니다:

1. **Model (get_model)**:
    - 지정된 모델명(예: `gemini/gemini-2.5-flash` , `anthropic/claude-3-5-sonnet`)과 프로바이더에 맞는 모델 인터페이스(LitellmModel 등) 인스턴스를 생성합니다.
2. **Environment (get_environment)**:
    - 로컬 실행 환경(LocalEnvironment)을 생성하며, 자동화에 지장을 주는 대화형 페이징(`PAGER=cat` , `TQDM_DISABLE=1` 등) 환경변수를 주입합니다.
3. **Agent (get_agent)**:
    - 모델과 환경 객체를 주입받아 대화형 에이전트(InteractiveAgent)를 생성합니다 (기본 모드: `confirm`).

5단계: 에이전트 실행 루프 (`agent.run(task)`)

- **컨텍스트 및 템플릿 렌더링**:
  - OS 정보(`system` , `release` , `machine`), 작업 내용(`task`), 동적 변수들을 Jinja2 템플릿에 주입하여 `system_template` 과 `instance_template` 을 렌더링하고 `messages` 리스트를 초기화합니다.
- **스텝 루프 반복 (`while True: agent.step()`)**:
  1. **한도 검사**: 스텝 수(`step_limit`), 비용(`cost_limit`), 제한 시간(`wall_time_limit_seconds`) 초과 여부를 먼저 체크합니다. 초과 시 대화형 터미널이면 사용자에게 한도 증액 여부를 묻습니다.
  2. **LLM 질의 (`query()`)**:
     - 현재까지의 메시지 히스토리를 LLM에 전달하여 모델의 추론(Reasoning) 및 Tool Call(`bash` 명령어)을 응답받습니다.
     - 모델 호출 횟수(`n_calls`)와 누적 비용(`cost`)을 계산하여 업데이트합니다.
  3. **사용자 인터랙션 및 승인 (Interactive Mode)**:
     - `confirm` 모드(기본값): 실행하려는 명령어를 터미널에 표시하고 사용자 승인(`y`), 거절(`n`), 수정 명령어 입력, `/yolo` 모드 전환 등을 대기합니다.
     - `yolo` 모드: 사용자 승인 없이 즉각 명령을 실행합니다.
     - `human` 모드 (`/u`): 사용자가 직접 터미널에 셸 명령을 입력하여 실행 결과를 에이전트 컨텍스트에 반영합니다.
  4. **도구 실행 (`execute_actions`)**:
     - LocalEnvironment에서 명령어를 독립 서브셸로 실행하고 `returncode` 와 표준 출력/에러 (`output`)를 수집합니다.
  5. **관측(Observation) 포맷팅 및 메시지 추가**:
     - 실행 출력이 10,000자를 초과하면 헤드와 테일만 남기고 생략(`elided_chars`) 처리합니다.
     - `format_observation_messages()` 를 통해 포맷팅된 관측 결과를 메시지 히스토리에 추가합니다.
  6. **궤적(Trajectory) 자동 저장**:
     - `~/.config/mini-swe-agent/last_mini_run.traj.json` 에 저장합니다.

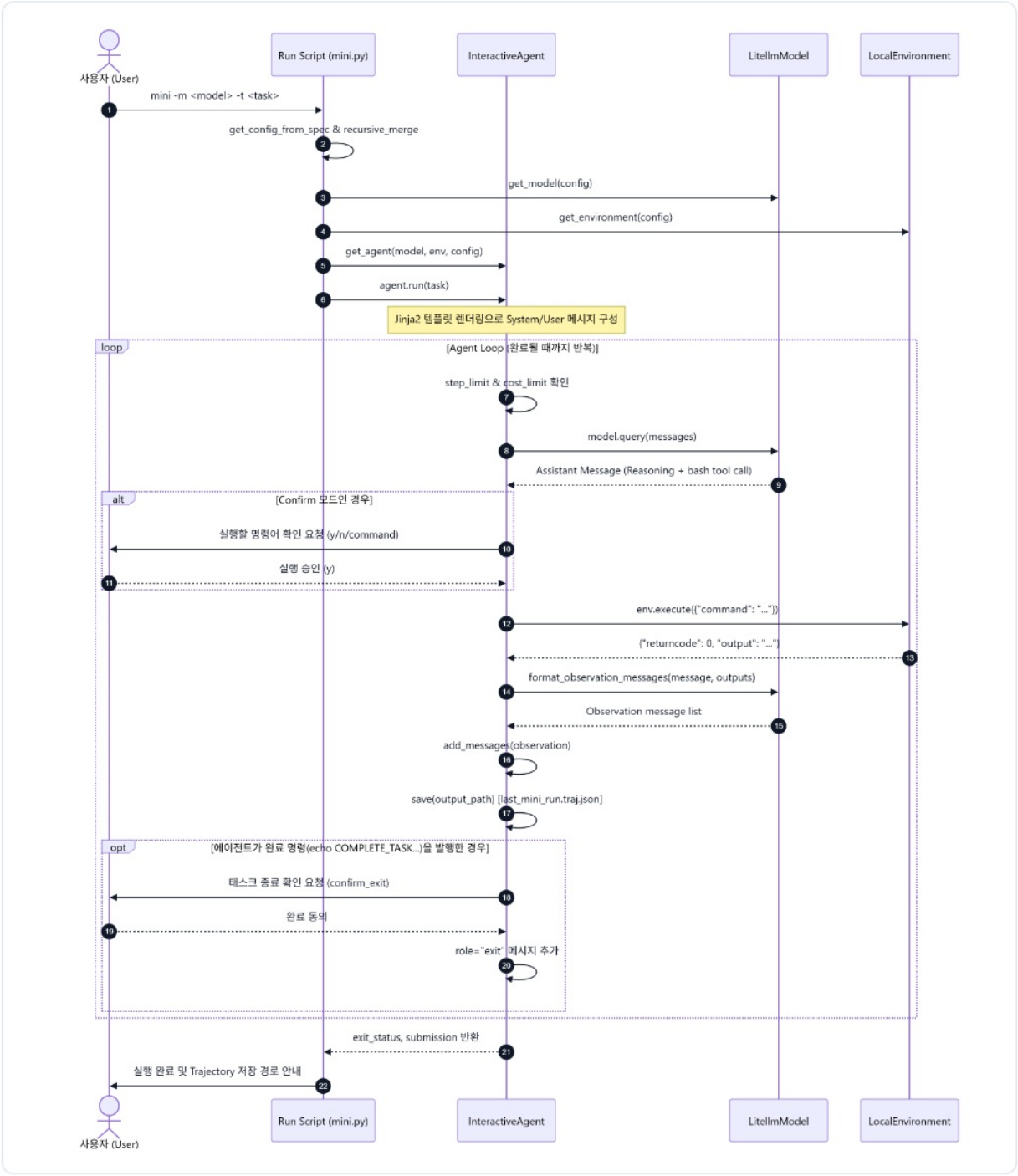
6단계: 작업 완료 및 종료 처리

- **완료 신호 감지**:
  - LLM이 해결 완료 신호인 `echo COMPLETE_TASK_AND_SUBMIT_FINAL_OUTPUT` 명령을 실행하면 완료 단계로 진입합니다.
  - `confirm_exit` 옵션에 따라 사용자에게 작업을 정말 종료할 것인지 확인한 후 `exit` 상태 메시지를 추가합니다.
- **최종 요약 및 리포트**:
  - 에이전트 루프가 종료되면 누적 스텝 수, 최종 사용 비용, 저장된 궤적 파일 경로를 터미널에 출력하고 실행을 마칩니다.

---

4. 에이전트 실행 루프 시퀀스 다이어그램 (Sequence Diagram)

사용자(User), CLI 실행기(Mini Run), 에이전트(Agent), 언어 모델(Model), 실행 환경(Environment) 사이의 호출 순서입니다.


사용자 (User)
Run Script (mini.py)
InteractiveAgent
LitellmModel
LocalEnvironment
mini -m <model> -t <task>
get_config_from_spec & recursive_merge
get_model(config)
get_environment(config)
get_agent(model, env, config)
agent.run(task)
Jinja2 템플릿 렌더링으로 System/User 메시지 구성
loop
[Agent Loop (완료될 때까지 반복)]
step_limit & cost_limit 확인
model.query(messages)
Assistant Message (Reasoning + bash tool call)
alt
[Confirm 모드인 경우]
실행할 명령어 확인 요청 (y/n/command)
실행 승인 (y)
env.execute({"command": "..."})
{"returncode": 0, "output": "..."}
format_observation_messages(message, outputs)
Observation message list
add_messages(observation)
save(output_path) [last_mini_run.traj.json]
opt
[에이전트가 완료 명령(echo COMPLETE_TASK...)을 발행한 경우]
태스크 종료 확인 요청 (confirm_exit)
완료 동의
role="exit" 메시지 추가
exit_status, submission 반환
실행 완료 및 Trajectory 저장 경로 안내

5. 상세 동작 메커니즘

5.1 설정 병합 메커니즘 (`recursive_merge`)

- 설정 우선순위:
  1. CLI 인자 플래그 (`--model` , `--agent-class` , `--yolo` , `--cost-limit` 등)
  2. 명시된 설정 파일 (`-c custom.yaml`)
  3. 기본 설정 파일 (mini.yaml)
- recursive_merge 함수를 통해 깊은 딕셔너리 병합이 수행되며, 미설정값(`UNSET`)은 상위 값으로 채워집니다.

5.2 템플릿 렌더링 시스템

- DefaultAgent는 `get_template_vars()` 를 통해 에이전트 상태, 환경 정보, 모델 정보, 작업 지시문을 취합합니다:
  - `{{task}}` : 사용자 해결 과제
  - `{{system}}` , `{{release}}` , `{{machine}}` : 실행 OS 환경 정보
  - `{{n_model_calls}}` , `{{model_cost}}` , `{{elapsed_seconds}}` : 동적 메트릭
- Jinja2의 `StrictUndefined` 를 사용하여 템플릿 변수 누락 시 즉각 오류를 발생시켜 문제를 조기에 방지합니다.

5.3 3가지 사용자 상호작용 모드 (`InteractiveAgent`)

1. **confirm 모드 (기본값)**:
   - LLM이 제안한 도구 명령어를 사용자 터미널에 표시하고 실행 여부를 묻습니다 (`y` , `n` , 직접 수정 명령어 입력, `/yolo` 모드 전환 등).
2. **yolo 모드 (`-y` / `--yolo`)**:
   - 확인 절차 없이 LLM의 명령어를 즉각 실행합니다. (CI/CD 환경 또는 자율 실행에 적합)
3. **human 모드 (`/u`)**:
   - 사용자가 직접 셸 명령을 입력하고 실행 결과를 컨텍스트에 추가하여 LLM을 보조합니다.

5.4 명령어 실행 및 관측 피드백

- LocalEnvironment:
  - 각 명령을 독립된 서브셸에서 실행(`subprocess.Popen`).
  - 환경변수(`PAGER=cat` , `TQDM_DISABLE=1` 등)를 주입하여 대화형 페이징으로 인한 행(hang) 현상을 방지합니다.
- Model Observation Template:
  - 출력 길이가 너무 길 경우(10,000자 초과), 앞단 5,000자와 뒷단 5,000자를 발췌하고 중간을 생략(`elided_chars`)하여 컨텍스트 윈도우 낭비를 막습니다.

5.5 작업 완료 및 궤적(Trajectory) 저장

- 작업 종료: 에이전트가 `echo COMPLETE_TASK_AND_SUBMIT_FINAL_OUTPUT` 명령을 실행하면 에이전트가 완료 상태로 진입합니다.
- 궤적 저장: 매 스텝마다 `Agent.save()` 가 호출되어 실행 과정 전체(메시지 기록, 비용, 사용 토큰 수, 환경 정보 등)를 JSON 형태로 저장합니다 (`~/.config/mini-swe-agent/last_mini_run.traj.json`).

---

6. 주요 파일 및 모듈 참조 링크

컴포넌트	파일 경로	설명
인터페이스/프로토콜	**init**.py	`Agent` , `Model` , `Environment` 기본 프로토콜 정의
CLI 진입점	mini.py	Typer 기반 `mini` 실행 명령어 구현
기본 에이전트	default.py	에이전트 핵심 루프, 템플릿 렌더링, 제약조건 검사
대화형 에이전트	interactive.py	confirm, yolo, human 모드 및 사용자 인터랙션 처리
LLM 모델 구현체	litellm_model.py	LiteLLM 기반 API 호출 및 Tool Call 파싱
로컬 환경	local.py	로컬 서브셸 명령어 실행 및 결과 반환
도커 환경	docker.py	도커 컨테이너 격리 실행 환경
기본 설정 파일	mini.yaml	시스템 프롬프트, 도구 규칙, 관측 템플릿 설정